Université Paris Nanterre
Université Sorbonne nouvelle
Institut national des langues
et civilisations orientales

pluriTAL

Master Traitement Automatique des Langues http://plurital.org

PluriTAL
Traitement statistique des données
2025-2026 (M1)

Projet :
Analyse automatique des avis de jeux : approche par classification binaire

Enseignant : Damien NOUVEL
Membres du projet : CHEN Peng, LIN Alice

Sommaire

Sommaire **2**
Introduction **3**
Démarche **3**
Dataset **4**
Analyse **4**
Scikit-learn 4
Naïve Bayes 4
SVM 6
WEKA 8
Prétraitement et configuration WEKA 8
VotedPerceptron 9
RandomForest 10
Discussion 11
Conclusion **11**
Documentation **12**

Introduction

Dans le cadre du cours de traitement statistique des données, nous avons réalisé un projet appliqué visant à mettre en pratique les différentes méthodes de classification étudiées en cours. Avec la croissance des plateformes de distribution de jeux mobiles comme TapTap, les utilisateurs publient chaque jour un grand nombre d'avis exprimant leur satisfaction ou insatisfaction. Ces données textuelles représentent une source riche d'information, mais leur volume important rend leur analyse manuelle difficile. Afin de répondre à cette problématique, nous nous intéressons à l'analyse automatique des avis de jeux vidéo à l'aide de méthodes de classification binaire. L'objectif est de déterminer automatiquement si un avis est positif (1) ou négatif (0) à partir de son contenu textuel. Pour cela, nous utilisons un jeu de données issu de la plateforme TapTap, composé d'avis de jeux mobiles en chinois, associés à un label de sentiment. Ce dataset constitue un cas d'étude pertinent pour appliquer des techniques de traitement statistique et de machine learning sur des données textuelles réelles.
Dans ce projet, nous avons mis en œuvre et comparé quatre modèles de classification étudiés en cours afin d'évaluer leurs performances sur cette tâche. L'objectif est d'analyser leur efficacité et de comprendre leurs différences dans un contexte concret d'application.

Démarche

Pour la mise en place de ce projet, nous avons tout d'abord déterminé un thème ainsi qu'une langue de travail. Après plusieurs recherches de jeux de données existants, notre choix s'est porté sur des avis de jeux vidéo rédigés en chinois. Le dataset sélectionné, disponible sur Kaggle, contient plusieurs types d'informations et de classifications possibles. Parmi ces différentes variables, la classification par sentiment nous a semblé la plus pertinente et la plus adaptée à notre objectif. L'auteure du dataset a déjà transformé cette variable en une classification binaire, nous avons donc conservé ce même encodage afin d'entraîner nos modèles de classification.

Une fois la base de données définie, nous nous sommes intéressés au choix des classificateurs. Plusieurs modèles ayant été étudiés en cours, tous présentant un intérêt dans ce type de problématique, nous avons réalisé une première phase de test à l'aide de l'outil WEKA afin de comparer leurs performances. À l'issue de ces tests, deux modèles ont été retenus pour la suite de notre étude : Random Forest et Voted Perceptron. Afin de nous familiariser avec l'utilisation des librairies Python, nous avons également sélectionné deux autres classificateurs issus de la bibliothèque Scikit-learn. Dans ce cadre, nous avons choisi d'expérimenter le classificateur Naïve Bayes ainsi que le modèle SVM (Support Vector Machine).

Dataset

Le jeu de données utilisé dans ce projet provient de la plateforme Kaggle et regroupe les avis d'utilisateurs sur des jeux vidéo mobiles publiés sur l'application TapTap. Il s'agit d'une plateforme très utilisée en Chine pour le partage et la distribution de jeux mobiles, où les utilisateurs peuvent laisser des commentaires et des notes sur les jeux auxquels ils jouent.

Le dataset est fourni sous forme de fichier CSV, et il contient 39 985 avis, répartis entre 13 632 avis négatifs et 26 353 avis positifs. Chaque avis correspond à un document. Les documents, qui correspondent à des commentaires d'utilisateurs, sont de longueur variable mais restent généralement courts. Ces données contiennent aussi plusieurs types d'informations associées à chaque avis, notamment : la note attribuée au jeu (de 1 à 5 étoiles), le contenu textuel du commentaire, le nombre de "likes" reçus, la date de publication, le modèle d'appareil utilisé, le nom du jeu concerné, ainsi qu'une variable de sentiment. Cette variable de sentiment constitue la cible de notre étude. Elle est définie sous forme binaire, où la valeur 0 correspond à un avis négatif et la valeur 1 à un avis positif. Cette transformation est basée sur la note attribuée par l'utilisateur et permet de simplifier le problème en une tâche de classification binaire.

Dans le cadre de ce projet, nous avons choisi de nous concentrer uniquement sur cette variable de sentiment, car elle est la plus pertinente pour une approche de classification et la plus adaptée aux modèles étudiés en cours. Le dataset constitue ainsi un cas concret d'application du traitement statistique de données textuelles en langue chinoise, dans un contexte réel d'analyse d'opinions utilisateurs.

Analyse

Scikit-learn

Pour chaque version de test, nous utilisons uniquement les colonnes *"review_content"* (commentaires) et *"sentiment"* (classification binaire). À la fin de chaque version, un test de prédiction est réalisé sur quatre textes de commentaires, répartis de manière équilibrée entre les classes 0 et 1, chacun contenant une expression de sentiment explicite ou implicite.

Naïve Bayes

			Accuracy	Précision	Rappel	F1	Macro avg
Naïve Bayes	**V1**	0	0.700	0.72	0.20	0.31	0.56
		1		0.70	0.96	0.81	
	V5	0	0.814	0.76	0.67	0.71	0.79
		1		0.84	0.89	0.86	
	V6 test	0	0.796	0.66	0.83	0.73	0.78
		1		0.90	0.78	0.83	

Tableau 1 : Les résultats obtenues des différentes versions de Bayes

Version 1 : Application de Naïve Bayes implémentée dans scikit-learn, lancée sans tokenisation ni nettoyage du corpus.

Le résultat obtenu est de 0,71, ce qui semble encourageant pour un premier essai. Nous avons ajouté un test de prédiction pour vérifier si le modèle a correctement appris à partir des données d'entraînement. À notre étonnement, il parvient également à traiter un texte en anglais, bien qu'entraîné sur des corpus chinois, avec des résultats corrects malgré un faible écart dans les scores de probabilité.

En nous concentrant sur le rappel et le F1, nous observons un écart important entre les deux classes. Le score de la classe négative (0) est très bas, tandis que celui de la classe positive (1) atteint presque 1. Malgré un score d'accuracy de 0,70, ce modèle ne peut pas être considéré comme fiable : ses classifications sont entièrement biaisées en faveur de la classe 1. La précision de la classe 0 est à 0,72 — les prédictions sont souvent correctes — mais son rappel à 0,20 indique que le modèle détecte très mal les vrais négatifs, prédisant « positif » dans la majorité des cas.

Lors du test de prédiction, seul le premier commentaire est correctement classé négatif (probabilité de 0,77 pour la classe 0). Pour les trois autres, les taux de probabilité sont identiques, ce qui trahit des prédictions aléatoires avec une préférence systématique pour la classe positive. Cette hésitation s'explique par l'absence d'indicateurs explicites dans les textes : seul le premier contient le terme 讨厌, qui guide clairement le modèle. Sans marqueur évident, celui-ci se rabat sur la classe majoritaire.

Avant d'arriver à la version 5, plusieurs tests intermédiaires ont été réalisés, mais les scores moyens oscillaient entre 0,60 et 0,63 — inférieurs à ceux de la version 1, contrairement à nos attentes

Version 5 : Ajout d'une tokenisation avec `jieba`, le corpus étant en chinois. Le modèle fonctionne toujours avec un ensemble d'entraînement et un ensemble de tests.

Schéma 1 : Token utilisé par NB avant et après prétraitement via jieba

Une augmentation globale des résultats est observée. Sans tokenisation, le modèle segmentait les textes en blocs de plusieurs caractères jusqu'à la prochaine ponctuation, produisant des tokens non significatifs. L'ajout de ce prétraitement rend les scores bien plus représentatifs des capacités réelles du modèle.

La classe positive conserve l'avantage en précision, rappel et F-mesure, en raison du déséquilibre du jeu de données — la classe 1 représentant presque le double de la classe 0. Malgré cela, les scores de la classe 0 sont plus stables : sa précision est à 0,76 et son rappel à 0,67, bien plus homogènes qu'en version 1. Les prédictions ne sont plus aléatoires ; le modèle a cette fois réellement « appris ». Le F1 macro moyen proche de 0,80 témoigne d'un modèle globalement équilibré.

Le test de prédiction confirme cette amélioration : les quatre textes sont correctement classifiés. Le modèle est très confiant pour les deux textes positifs (probabilité négative à peine supérieure à 0,10), mais plus hésitant pour les négatifs — le texte 2 n'obtenant que 0,66 de probabilité pour la classe 0, faute de marqueurs explicites et d'échantillons similaires dans le jeu de données.

Version 6 : Ajout d'un ensemble de développement (*dev*). Le jeu de données est divisé en trois ensembles : entraînement (80 %), test et dev (10 % chacun). Le dev permet de sélectionner la valeur optimale du paramètre de lissage alpha.

La meilleure valeur retenue est alpha = 0,1, ce qui indique un jeu de données très informatif ne nécessitant pas un lissage important. Le score d'accuracy sur le test est de 0,796, cohérent avec celui du dev (0,784), attestant d'une bonne stabilité. Le rappel de la classe 0 progresse fortement à 0,83, mais sa précision régresse de 0,1, signalant un risque de sur-prédiction des négatifs. La classe 1 conserve de bons scores, bien que son rappel baisse de 0,11.

Malgré ces nuances, les tests de prédiction révèlent une amélioration concrète : le modèle est moins hésitant sur la classe 0, le taux du texte 2 passant de 0,66 à 0,77. Bien que `class_weight=balanced` ne se soit pas traduit par une nette progression dans les métriques brutes, le déséquilibre est moins prononcé en pratique. Pour aller plus loin, il faudrait intervenir directement sur le jeu de données en renforçant manuellement le poids de la classe négative.

SVM

			Accuracy	Précision	Rappel	F1	Macro avg
SVM	**V1**	0	0.808	0.75	0.66	0.70	0.78
		1		0.83	0.89	0.86	
	V2	0	0.780	0.65	0.78	0.71	0.77
		1		0.87	0.78	0.82	
	V3	0	0.789	0.71	0.65	0.68	0.76
		1		0.83	0.86	0.84	
	V4 dev V4 test	0	0.795	0.67	0.79	0.73	0.78
		1		0.88	0.80	0.84	

			Accuracy	Précision	Rappel	F1	Macro avg
		0	0.783	0.65	0.79	0.71	0.77
		1		0.88	0.78	0.83	
	V4.1 dev V4.1 test	0	0.798	0.73	0.65	0.69	0.77
		1		0.83	0.87	0.85	
		0	0.790	0.71	0.65	0.68	0.76
		1		0.83	0.86	0.84	

Tableau 2 : Les résultats obtenus des différentes versions de SVM

Version 1 : Utilisation de `LinearSVC` de scikit-learn. Prétraitement avec `jieba` et kernel linéaire.

Le kernel linéaire a été retenu, recommandé pour les données textuelles selon la documentation officielle, avec l'argument `probability=True` pour l'évaluation. L'accuracy est de 0,80, mais un déséquilibre similaire à Naïve Bayes est présent. La classe 0 affiche une précision de 0,75 mais un rappel de 0,66 — de nombreux vrais négatifs sont manqués. La classe 1 atteint des scores proches de 0,90, en grande partie grâce à son surpoids dans le jeu de données (F1 macro = 0,78).

Lors des tests de prédiction, trois textes sur quatre sont bien classifiés. Le texte 2, dépourvu de marqueurs explicites, est correctement classé mais avec seulement 0,60 de probabilité — contre 0,91 pour le texte 4 en classe positive, confirmant le biais du modèle. Pour améliorer les résultats, une gestion du déséquilibre et l'ajout de stopwords seraient nécessaires.

Version 2 : Prétraitement enrichi (conservation des caractères chinois uniquement, ajout de stopwords), SVC avec gestion du déséquilibre via `class_weight=balanced`.

Cette version perd 0,02 en accuracy, mais la détection des négatifs s'améliore et le modèle est plus équilibré. La précision de la classe 0 diminue au profit d'une hausse de son rappel, tandis que la classe 1 voit sa précision progresser mais son rappel baisser légèrement. La gestion du déséquilibre a rempli son rôle.

Les tests de prédiction le confirment : le texte 2 passe de 0,65 à 0,73 de probabilité pour la classe 0, le modèle gérant mieux les textes sans indicateurs explicites. Le déséquilibre demeure toutefois visible — 0,93 pour le texte 4 en classe positive contre 0,74 pour le texte 2 en classe négative.

Version 3 : Comparaison des kernels linéaire et RBF (gamma *scale* et *auto*) avec différents paramètres C.

Le module a été configuré pour choisir librement entre un kernel linéaire et RBF, et le choix retenu est affiché en première ligne des résultats. À notre étonnement, c'est le kernel RBF qui est sélectionné ; pourtant, les résultats de cette version 3 s'approchent de ceux de la version 1. RBF ne s'avère pas plus performant sur notre jeu de données : le biais en faveur de la classe 1 réapparaît, le rappel de la classe 0 retombant à 0,65 — soit 0,01 de moins que la version 1. Pour le paramètre C, le module compare les valeurs 0,1, 1 et 10, et retient C = 10, valeur élevée qui pousse le modèle à coller davantage aux données

d'entraînement afin de minimiser les erreurs. Ce choix entraîne un biais vers la classe majoritaire, soit la classe 1.

Les tests de prédiction entre les versions 1 et 3 sont quasi identiques : le texte 2 reste hésitant, tandis que les textes à classification explicite affichent de meilleurs taux de probabilité qu'en version 1. Pour le texte 4, appartenant à la classe majoritaire, le modèle est très confiant, même en l'absence d'indicateurs explicites.

Version 4 : Ajout d'un ensemble de développement (*dev*), avec une division train / dev / test (60 % / 20 % / 20 %). Premier test avec le kernel linéaire uniquement, second test avec choix entre linéaire et RBF (sur la base des résultats du dev en version 3, le meilleur kernel semblant être RBF).

Kernel linéaire : Le modèle est stable entre dev et test (baisse de 0,01 en accuracy). La classe 0 obtient 0,79 en rappel, mais sa précision diminue, traduisant une confusion entre vrais positifs et faux négatifs. La classe 1 reste très performante. La valeur de C retenue est faible, favorisant la généralisation et l'équilibre. Les tests de prédiction sont tous corrects, avec des taux comparables à la version 2.

Ajout du kernel RBF : Le modèle préfère RBF avec un C plus élevé, et la stabilité entre dev et test est excellente. Cependant, le kernel linéaire reste plus performant pour la détection de la classe 0, tandis que RBF se montre plus conservateur et biaisé vers la classe 1.

Cette dernière version est la plus performante pour la classe positive, mais pour une classification équilibrée, le kernel linéaire demeure le choix le plus pertinent. Le réglage des hyperparamètres via le dev a surtout permis de confirmer la stabilité du jeu de données et la bonne généralisation des modèles.

WEKA

Prétraitement et configuration WEKA

Nous n'avons pas utilisé directement le script de vectorisation fourni par M. Dupont. À la place, nous nous sommes inspirés du script "corpus2arff.py" vu en cours afin de développer notre propre pipeline "preprocess.py", qui génère des fichiers ARFF (train/dev/test) compatibles avec WEKA.

En raison de la taille importante du corpus et des limitations de performance de l'ordinateur, nous n'avons pas pu charger l'ensemble du jeu de données dans WEKA, ni effectuer l'entraînement complet. Nous n'avons utilisé que 50 % des données originales. Un prétraitement a été appliqué : conservation des champs 'review_content' et 'sentiment', suppression des valeurs manquantes, nettoyage des caractères non pertinents (uniquement chinois, lettres et chiffres), tokenisation avec `jieba`, suppression des stopwords chinois, puis conversion au format ARFF.

Ensuite, pour assurer la cohérence du vocabulaire entre les ensembles d'entraînement, de développement et de test, nous avons utilisé dans WEKA un 'FilteredClassifier' combiné à 'StringToWordVector' afin de réaliser la vectorisation des textes. Nous avons constaté que le corpus d'entraînement initial était déséquilibré, avec 10 505 avis positifs contre 5 461 avis négatifs. Pour corriger ce problème, nous avons

appliqué le filtre ‘SpreadSubsample’ (distributionSpread = 1.0), ce qui a permis d’obtenir un nombre égal d’exemples (5461) pour chaque classe.

Sur ce corpus rééquilibré, nous avons entraîné les modèles RandomForest et VotedPerceptron, puis évalué leurs performances sur le corpus de développement avec les paramètres par défaut de WEKA. Les résultats détaillés sont disponibles sur GitHub.

Modèle	Corpus équilibré	Accuracy	Weighted F1	Recall négatif	Recall positif	F1 négatif
VotedPerceptron	Non	77.52%	0.769	0.591	0.874	0.648
RandomForest	Non	78.22%	0.774	0.569	0.897	0.647
VotedPerceptron	Oui	73.71%	0.742	0.726	0.743	0.659
RandomForest	Oui	76.77%	0.771	0.744	0.780	0.692

Les résultats montrent que les modèles entraînés sur un corpus non équilibré favorisent la classe positive, avec un recall plus faible pour la classe négative. La classe minoritaire est donc moins bien reconnue. Après rééquilibrage des données, le recall et le F-mesure de la classe négative augmentent nettement, ce qui améliore la détection des avis négatifs. L’impact du déséquilibre des classes sur les performances est ainsi clairement visible.

Ce comportement s’explique par la tendance des modèles à privilégier la classe majoritaire lorsqu’ils sont entraînés sur des données déséquilibrées. L’accuracy globale peut alors donner une impression de meilleures performances, alors que les résultats sur la classe minoritaire restent faibles. Dans notre tâche d’analyse de sentiments, les avis négatifs étant importants, leur mauvaise classification a un coût plus élevé. Il ne suffit donc pas de se baser uniquement sur l’accuracy. Même si l’accuracy diminue légèrement après équilibrage, la perte reste limitée, tandis que la reconnaissance de la classe négative s'améliore nettement. Pour cette raison, le réglage des paramètres ainsi que l’entraînement final ont été réalisés sur le corpus équilibré.

VotedPerceptron

Pour VotedPerceptron, les paramètres ont été ajustés à partir des résultats obtenus sur le corpus de développement (voir GitHub). Nous avons testé différentes configurations en faisant varier le nombre d’itérations (3, 5, 100), l’exposant (1, 2, 3) ainsi que la taille du vocabulaire (1000 et 5000).

Expérience	I	E	W	Accuracy	Weighted F1	pos F1	neg F1
pc-1	3	1.0	1000	74.91%	0.753	0.798	0.668
pc-2	5	1.0	1000	74.86%	0.752	0.798	0.667
pc-3	100	1.0	1000	74.86%	0.752	0.798	0.667
pc-4	5	2.0	1000	75.36%	0.757	0.803	0.671
pc-5	5	3.0	1000	73.66%	0.741	0.787	0.656
pc-6	5	2.0	5000	74.76%	0.751	0.796	0.668

Les résultats montrent que le VotedPerceptron est globalement stable, avec des performances assez proches selon les configurations (Accuracy entre 73.66% et 75.36%, Macro F1 entre 0.741 et 0.757).

Le nombre d'itérations a peu d'impact sur les résultats, ce qui suggère une convergence rapide du modèle. Le paramètre E joue un rôle plus visible : E = 2 donne les meilleurs scores, tandis que E = 3 entraîne une légère baisse des performances. La taille du vocabulaire a un effet limité, mais 1000 mots fonctionne légèrement mieux que 5000, probablement parce que la représentation est plus compacte et moins bruitée. Nous retenons donc la configuration I = 5, E = 2.0 et W = 1000, qui offre le meilleur compromis global.

Une évaluation finale est ensuite effectuée sur le corpus de test.

Model	Config	Accuracy	Weighted F1	pos F1	neg F1
VotedPerceptron	I=5/E=2/W=1000	75.24%	0.758	0.803	0.668

Sur le corpus de test, le modèle obtient des performances globalement stables (accuracy = 75.24 %, F1 pondéré = 0.758), avec de meilleurs résultats pour la classe positive que pour la classe négative.

RandomForest

Pour RandomForest, les paramètres ont été ajustés à partir des résultats obtenus sur le corpus de développement (voir GitHub). Nous avons testé le nombre d'arbres (50, 100, 200) ainsi que la taille du vocabulaire (1000 et 5000).

Expérience	Trees	Vocab	Accuracy	Weighted F1	pos F1	neg F1
rf-1	50	1000	76.72%	0.770	0.813	0.690
rf-2	100	1000	76.77%	0.771	0.814	0.692
rf-3	200	1000	77.07%	0.774	0.815	0.697
rf-4	200	5000	78.22%	0.785	0.826	0.709

Les résultats montrent un modèle globalement stable, avec des performances proches selon les configurations (Accuracy entre 76.72 % et 78.22 %, Weighted F1 entre 0.770 et 0.785).

Le nombre d'arbres a un effet limité mais positif, avec une légère amélioration entre 50 et 200 arbres. La taille du vocabulaire joue un rôle : 5000 features donnent de meilleurs résultats que 1000, contrairement au VotedPerceptron. Cela peut s'expliquer par la robustesse de RandomForest face aux données de grande dimension et au bruit grâce à son mécanisme de sélection aléatoire des variables. Nous retenons donc la configuration 200 arbres et 5000 features comme meilleure configuration.

Une évaluation finale est ensuite effectuée sur le corpus de test.

Model	Trees	Vocab	Accuracy	Weighted F1	pos F1	neg F1
RandomForest	200	500	78.15%	0.785	0.829	0.698

Sur le corpus de test, le modèle obtient des performances globalement stables (accuracy = 78.15 %, F1 pondéré = 0.785). Nous observons de meilleurs résultats pour la classe positive que pour la classe négative, mais les performances restent correctes pour les deux classes, sans déséquilibre majeur.

Discussion

Il existe une différence de complexité entre ces deux modèles : le modèle linéaire VotedPerceptron s'entraîne très rapidement (1,58 seconde), tandis que le RandomForest nécessite un temps de calcul beaucoup plus important (581,55 secondes).

Les deux modèles présentent une tendance similaire, avec de meilleures performances sur la classe positive que sur la classe négative. Globalement, RandomForest obtient de meilleurs résultats que VotedPerceptron, avec une performance plus élevée et plus stable sur l'ensemble des métriques.

Cette différence s'explique par un ensemble de facteurs liés à la fois aux algorithmes et à la nature de la tâche. Le VotedPerceptron, en tant que modèle linéaire, reste limité dans la modélisation de structures linguistiques complexes ou implicites comme l'ironie. Par ailleurs, la tâche de classification de sentiments et la nature des données en chinois peuvent également contribuer à cette asymétrie, les expressions négatives pouvant être plus variées et moins explicites que les expressions positives.

À l'inverse, RandomForest utilise une approche non linéaire par ensemble, ce qui lui permet de mieux gérer cette variabilité et de capturer certaines interactions entre les mots.

Conclusion

Ce projet avait pour objectif de mettre en œuvre et de comparer plusieurs classifieurs — Naïve Bayes, SVM, Voted Perceptron et Random Forest — dans le cadre d'une tâche de classification binaire portant sur l'analyse de polarité d'avis de jeux mobiles rédigés en chinois, à partir du corpus TapTap.

Les expériences ont montré que les performances des modèles sont fortement influencées par la qualité du prétraitement des données. La segmentation adaptée au chinois (jieba), la suppression des mots vides ainsi que la gestion du déséquilibre entre classes se sont révélées déterminantes. Sans ces étapes, les modèles tendaient à privilégier la classe majoritaire et produisaient des prédictions peu fiables pour les avis négatifs.

Après prétraitement, l'ensemble des modèles atteignent des performances cohérentes et globalement satisfaisantes, avec une accuracy comprise entre 0.75 et 0.80 selon les configurations. Les modèles linéaires (Naïve Bayes, SVM, VotedPerceptron) restent efficaces mais montrent des limites dans la détection des sentiments implicites ou nuancés, notamment dans les avis négatifs. Le SVM linéaire apparaît comme une approche robuste lorsqu'il est correctement paramétré, tandis que le noyau RBF n'apporte pas d'amélioration significative dans ce contexte.

Les expériences réalisées sous WEKA confirment également l'impact du déséquilibre des classes : le rééquilibrage améliore la détection des avis négatifs, mais peut légèrement réduire l'accuracy globale. Le VotedPerceptron montre un comportement stable mais limité par sa nature linéaire, tandis que Random Forest offre une meilleure capacité à modéliser des relations non linéaires entre les mots.

Le principal point faible du modèle reste la difficulté à capturer les sentiments négatifs implicites, sarcastiques ou dépendants du contexte. Même les meilleurs modèles restent limités par une représentation bag-of-words relativement simple.

En conclusion, ce travail montre que des méthodes classiques de machine learning, combinées à un prétraitement adapté, permettent d'obtenir des résultats satisfaisants pour la classification de sentiments en chinois. Toutefois, des améliorations sont possibles, notamment l'utilisation de représentations textuelles plus riches (TF-IDF, n-grammes, embeddings), ainsi que l'exploration de modèles neuronaux pré-entraînés capables de mieux capturer les nuances sémantiques.

Documentation

Jeux de données :
https://www.kaggle.com/datasets/karwinwang/taptap-mobile-game-reviews-chinese/data

Notre dépôt github : https://github.com/PengCHEN2001/Classifieur

Naïve Bayes : https://scikit-learn.org/stable/api/sklearn.naive_bayes.html

SVM : https://scikit-learn.org/stable/api/sklearn.svm.html